AI INDUSTRY · INSIGHT REPORT

Code Zero

코드가 0이 된 자리에서
무엇이 가치를 갖는가

2026. 3

EXECUTIVE SUMMARY

2026년 초, AI 산업이 분명한 전환점을 지났다. 에이전트가 자율적으로 프로덕트를 만들고, 벤치마크가 포화되며, 40일 만에 100만 줄의 코드가 완성되는 현실이 동시에 펼쳐지고 있다. **코드의 가치는 0으로 수렴**하고 있고, 그 빈자리를 무엇으로 채울 것인가가 유일한 질문이 되었다. 본 리포트는 이 전환의 실체를 6개 축으로 분석하고, 낙관과 비관 양쪽을 함께 검토한 뒤, 실행 가능한 원칙을 제시한다.

목 차

01 에이전트는 조직이 되었다

02 하네스 경제학

03 RL 모노컬처

04 마지막 해자 — 암묵지

05 빈자리의 경제학

06 가속의 안쪽에서 살아남기

01

에이전트는 조직이 되었다

명령-실행 구조에서 자율 팀 구조로의 이행

반년 전까지 에이전트 코딩의 주류는 단순한 명령-실행 루프였다. 하나의 에이전트에게 지시를 내리고, 결과를 받고, 피드백을 주는 구조. 지금은 완전히 다른 풍경이다. **16개의 에이전트가 병렬**로 돌아가면서 C 컴파일러를 만들어내고, 수천 단위의 에이전트가 하나의 코드베이스 위에서 동시에 작업한다.

핵심 구조 변화: 초기에는 메인 에이전트가 서브 에이전트를 일일이 관리하는 구조로, 관리자의 부하가 병목이었다. 지금은 **공유 태스크 리스트**를 통해 팀원들이 자율적으로 통신하고, 서로의 작업을 조율하는 구조로 진화했다. 리드 에이전트는 방향만 잡아주고, 실행은 팀이 알아서 한다. HBR에서 리더십을 다룰 때 늘 강조하는 것 — 의도를 이끌어내고, 명시하고, 이해시키는 일 — 이 정확히 에이전트 팀에서도 핵심 과제가 되었다.

두 학파: 자율 반복 vs 사람 개입

자율 반복(Ralph Loop)은 스펙을 정하고 될 때까지 에이전트가 반복하는 접근이다. **사람 개입(Human In The Loop)**은 에이전트와 실시간으로 티키타카하며 조율하는 접근이다. 스펙이 명확하면 전자, 탐색이 필요하면 후자. 실전에서는 병행이 현실적이다. 어느 쪽이든 **컨텍스트 보존**이 핵심 과제이며, 에이전트 수십 개가 병렬로 돌아갈 때의 **조향 가능성(steerability)**과 **관측 가능성(observability)**이 에이전트 시대의 근본 인프라다.

직교적 설계: 최소 단위의 조합

RGB가 3축으로 모든 색을 만들듯, 서로 독립적인 최소 기능 단위를 조합해 무한한 결과를 만드는 설계 철학이 부상하고 있다. 범용 프로토콜에 의존하는 대신, 에이전트가 필요한 소프트웨어를 스스로 만들어가는 최소주의 접근이 현장에서 강력한 결과를 내고 있다.

> 취향, 판단, 방향성은 인간에게서 나왔지만 AI는 빠른 반복과 탐색을 위한 강력한 force multiplier였다.

02

하네스 경제학

모델이 NAND라면, 하네스는 Firmware다

40일간 130억 토큰을 투입해 100만 줄의 프로덕트를 완성한 실전 사례가 있다. 8대의 PC에서 에이전트를 병렬로 돌렸고, 과거 3년이 걸렸을 작업이 40일로 압축되었다.

40일	130억	100만+
프로덕트 완성 기간 (과거 기준 3년)	투입 토큰 수	생성 코드 라인 수

모델이 아니라 하네스가 승부를 가른다

에이전트 코딩 도구의 핵심 경쟁력은 모델 자체가 아니다. 모델을 감싸서 결정론적으로 동작시키는 **하네스(harness) 코드**에 있다. 동일한 모델이라도 어떤 하네스에 붙이느냐에 따라 결과가 극적으로 달라진다. 심지어 다른 회사의 모델도 좋은 하네스에 연결하면 훨씬 나은 결과를 낸다. SSD에 비유하면 — 최고의 NAND를 탑재해도 Firmware가 부족하면 성능이 안 나온다. 모델이 NAND이고, 하네스가 Firmware인 셈이다.

성능 = Model × Harness × Pilot

세 변수 중 하나라도 0이면 전체가 0이다. 최고의 모델을 써도 하네스가 부족하면 성능이 안 나오고,
둘 다 갖춰도 Pilot이 무엇을 만들어야 하는지 정의하지 못하면 의미가 없다.

토큰 경쟁력 = 조직 경쟁력

모두가 AI로 코딩하는 세상에서 승부를 가르는 것은 두 가지다. 첫째, **같은 결과를 더 적은 토큰으로 내는 것**. 성능이 필요한 곳만 깊이 생각하고 단순 작업은 가볍게 처리하는 adaptive thinking budget이 핵심이 된다. 둘째, **같은 토큰을 더 빠르게 생성하는 것**. 기존 코드 생성 속도의 5~10배에 해당하는 초고속 inference가 실질적 경쟁 우위다.

결과물이 아니라 생성 장치를 만든다

이메일 하나를 쓰는 게 아니라, 이메일을 자동으로 생성할 수 있는 프로젝트와 하네스를 셋업하는 것이 새로운 방식이다. soul document를 기반으로 컨텍스트를 쌓고, 자동화 파이프라인을 구축한다. 핵심은 에이전트에게 스펙을 바로 던지는 게 아니라, **컨텍스트를 같이 만들어가는 선문답의 과정**이다.

03

RL 모노컬처

답지는 공개되었다 — 문제는 주방 설비다

최신 모델이 METR 벤치마크에서 **14시간**을 기록하며 벤치마크 자체가 포화 상태에 이르렀다. 인간의 8시간 근무를 훨씬 초과하는 수준으로, 'AI가 사람 개입 없이 얼마나 오래 일할 수 있는가'라는 질문 자체가 의미를 잃어가고 있다. 모델 릴리즈는 1시간 간격으로 경쟁사의 최상위 모델이 발표되고, 0.1 단위 버전업이 트렌드가 되었다.

모든 혁신이 RL로 수렴한다

현재 프론티어 모델의 **거의 모든 혁신이 RL(강화학습)**에 관련되어 있다.

> ① **효율** — Sparse Attention, 비동기 RL 등을 통해 같은 연산으로 더 많이 학습
> ② **안정성** — objective 함수 수정으로 학습 도중 모델이 이상해지지 않게
> ③ **환경 다양성** — 코딩뿐 아니라 과학, 수학, 도구 사용 등으로 RL 적용 범위 확장

"방법에 대한 답을 찾았으니, 여러 문제에 적용하면 된다." 지난 1년간 이 진단은 정확했다.

그러나 — 레시피 공개 ≠ 격차 해소

> 레시피는 공개되어 있다. 그러나 **주방 설비의 격차는 압도적**이다. "기본기"라는 것 — 좋은 데이터, 안정적 인프라, 충분한 연산 — 은 결국 **수조 원 단위의 자본과 수년간의 경험치**다. 레시피를 안다고 해서 미슐랭 3스타 레스토랑을 차릴 수 있는 것은 아니다.

후발주자에게 기회가 있는 것은 맞다. 중국 기업들이 독자적으로 동일한 방법에 도달했다는 사실이 증명한다. 그러나 "기본기만 잘하면 따라잡을 수 있다"는 결론은 과도하게 낙관적이다. 기본기를 쌓는 데 필요한 자원의 격차가 핵심 장벽이며, 이 장벽은 비밀 레시피보다 오히려 넘기 어렵다.

04

마지막 해자 — 암묵지

도메인의 95%가 증발한 뒤에 남는 것

프론티어 모델이 거의 모든 도메인에서 **95% 이상을 커버**하게 되면서 '도메인 전문성'이라는 해자는 거의 소멸했다. 남은 5%는 **암묵지(tacit knowledge)**다. 문서화되지 않은, 경험에서 비롯된 의사결정 노하우. 철강 장인이 배합 시점을 아는 것, 법률 전문가가 떠올린 전략 하나로 판을 바꾸는 것 — 결정적 문제 해결은 이 최상위 암묵지에서 나온다.

암묵지를 모델이 쓸 수 있게 만드는 법

경로	방법	특성
A. 텍스트 전량 기술	모든 경우를 글로 써서 모델에 학습시킴	가능하지만 규모가 크고, 프론티어 모델이 이미 공개 암묵지 대부분을 습득
B. Context Graph	객체와 관계로 압축 표현, graph RAG/ontology로 모델에 공급	텍스트보다 훨씬 작은 크기로 동일 효과. 자사 고유 데이터에 적합

추상적 조언을 넘어서 — 실전 적용

"암묵지를 문서화하세요"는 "운동하세요" 수준의 조언이다. 구체적으로 보면: 주식 트레이딩의 경우, "이란 뉴스 → 방산주 선행 → 유가 동행 → 반도체는 나스닥 T-1 상관"이라는 판단 체계를 3레이어 시그널 아키텍처로 만들어 에이전트에 탑재하는 것이 context graph의 실전 버전이다. 법률이라면, 특정 분쟁 유형에서 "이 조항이 있으면 이 전략이 유효하다"는 판례 기반 의사결정 트리를 ontology로 구조화하는 것이다. 핵심은 **나의 특정 도메인 × 나의 특정 판단 체계 × 나의 특정 도구**라는 3중 교차점에서만 해자가 성립한다는 점이다.

타임 갭 × 암묵지 갭 = 비즈니스 기회

타임 갭은 월→주 단위로 압축 중. 암묵지 갭만이 유일한 해자. 이 교집합만이 빅테크의 중력을 이길 수 있는 영역이다.

05

빈자리의 경제학

코드가 0이 된 자리를 누가 채우는가

소프트웨어 산업은 세 번째 대변혁기에 있다. 1차는 천공카드→키보드, 2차는 패키지→웹/모바일, 3차인 지금은 코드 작성→**의미 전달(자연어) 기반 개발**로의 전환이다. 미래 소프트웨어는 AI 코어 엔진(80%) + 결정론적 제어 레이어(10%) + 인터페이스(10%)로 구성될 것이다.

수혜자 구조 — 문과/이과가 아니다

"문과가 2순위 수혜자"라는 단순화는 오해를 낳는다. 실제 축은 **"문제를 정의할 수 있는 사람 vs 지시받은 것만 실행하는 사람"**이다.

유형	특성	현재 상태
1순위 수혜자	깊은 엔지니어링 경험 + 사업가 감각. 문제 정의→빌딩→사업화 전 과정을 AI로 가속	20~30년 경력 시니어들이 자기 harness를 만들며 독주 중
2순위 수혜자	코딩은 못하지만 **evaluation metric을 설정할 수 있는** 도메인 전문가	시작점과 평가 기준만 세우면 에이전트가 답을 찾아옴
피해 구간	코딩 스킬만 보유, 고객 개발 부재	솔루션은 양산하지만 쓸 사람 없는 AI slop 넘쳐남

과도기의 현실적 고통을 직시해야 한다

> 산업혁명 때도 "새로운 직업이 생길 것"이라고 했고 실제로 생겼다. 그러나 그 과도기에 **한두 세대가 처참하게 고생했다**. 지금은 변화 속도가 훨씬 빠르기 때문에 충격도 더 클 수 있다. "정부가 메꿔줄 것"이라는 전제도 현실의 정부 역량을 보면 낙관하기 어렵다. "엔터테인먼트 경제에서 새 직업이 탄생한다"는 가장 약한 가설이다. 거시적 낙관에 기대지 말고, **개인 단위에서 지금 당장 실행 가능한 전환**에 집중하는 것이 현실적이다.

06

가속의 안쪽에서 살아남기

9의 행진, 그리고 바이오 토큰

9의 행진 (March of Nines)

90%→99%→99.9%→99.99% — **각 단계에 들어가는 노력의 크기는 동일하다.** AI가 해주는 것은 90~99% 구간이다. 컴파일러 분야의 거장이 AI가 만든 C 컴파일러를 심층 리뷰한 결과: "교과서적 구현으로서 가치가 있지만, 새로운 추상화를 발명하지는 못했다." 나머지 nines를 채우는 곳에서 진정한 차별화가 만들어진다.

> 구현이 쉬워질수록 진짜 혁신이 들어설 공간은 더 커진다. 만들기가 쉬워질수록, 무엇이 만들 가치가 있는지를 결정하는 일이 더 어려운 문제가 된다.

FOMO는 구조적이다

최전선에서 변화를 이끄는 사람들조차 "매일같이 힘들다", "제정신이기가 어렵다"고 고백한다. 이것은 역량의 문제가 아니라, **가속도 자체가 증가하는 구조**에서 발생하는 자연스러운 반응이다.

바이오 토큰 — 이 리포트에서 가장 중요한 주제일 수 있다

에이전트 코딩은 모바일 가챠와 비슷한 도파민 구조를 갖고 있다. 시켜보고→결과 확인→더 시키고. 이 루프에 빠지면 **흰머리가 늘고, 수면이 줄고, 인지 부하가 폭증**한다. "40일 동안 3년치 늙었다"는 증언은 과장이 아니다.

> 생산성 도구가 사람을 착취하는 구조로 변하는 것은 AI 시대의 **가장 현실적인 위험**이다. 코드의 가치가 0이 되는 것보다, 인간의 인지 자원(바이오 토큰)이 고갈되는 것이 더 시급한 문제일 수 있다.

존댓말	내 것	MVK	휴식
AI에게 쓰는 말투가 사람에게 전이된다	남의 skill 복사가 아닌 내 일의 위탁에서 시작	모르는 채로도 옳은 방향으로 갈 최소 지식	도파민 루프를 끊고 바이오 토큰을 회복

결론: 실행 가능한 원칙

01 하네스를 구축하라.

soul document부터 시작해 내 업무를 위탁하는 자동화 파이프라인을 만든다. 남의 도구가 아니라 내 워크플로우에 맞는 것을 직접 설계한다.

02 암묵지를 구체적으로 명시화하라.

나의 특정 도메인에서, 나의 특정 판단 체계를, 나의 특정 도구에 녹여넣는다. context graph든 시그널 아키텍처든, 형태는 상관없다.

03 문제 정의자가 되라.

코딩이 아니라 문제 정의와 evaluation metric 설계가 핵심 역량이다. 고객 개발이 제품 개발보다 중요하다.

04 토큰 효율성을 경영하라.

adaptive thinking budget, 고속 inference, 병렬 에이전트 활용으로 동일 비용에서 최대 생산성을 뽑아낸다.

05 99%에 안주하지 마라.

AI가 해주는 90~99%에 만족하지 말고, 나머지 nines를 채우는 데 집중하라. 교과서적 구현은 AI가, 새로운 추상화의 발명은 인간의 몫이다.

06 바이오 토큰을 사수하라.

이것이 가장 중요한 원칙일 수 있다. AI의 도파민 루프에 휘둘리지 말고, 인지 자원을 의식적으로 관리하라. 기술은 도구이지 주인이 아니다.

정답은 없지만 태도는 있다 — 호기심을 유지하고, 내 경험을 쌓아가되,
떠밀려 가는 와중에도 놓치고 싶지 않은 것을 놓치지 않는 것.